

Uma Heurística de Ramificação Orientada pela Estrutura do Grafo para o Problema da Mochila com Conflitos

Matheus Kennedy T. Santos¹

¹Instituto de Computação – Universidade Federal de Alagoas (UFAL)
Maceió – AL – Brasil

Abstract. *The Knapsack Problem with Conflicts (KPC) is an NP-hard extension of the classical knapsack problem that introduces disjunctive constraints among items, significantly increasing the complexity of exact solution methods. This paper proposes a branching heuristic, named S_i , which incorporates topological attributes of the conflict graph, such as vertex degree, into the decision process of the Branch-and-Bound algorithm. The heuristic combines a topological factor with an adaptive economic term that is sensitive to the density of the conflict graph. Computational experiments conducted under two distinct methodological settings demonstrate that the proposed approach reduces the number of explored nodes by an average of 44.31% compared to the classical Most Fractional branching rule. The results confirm the effectiveness of exploiting topological information to induce earlier pruning while preserving solution optimality, suggesting that graph-aware branching strategies constitute a promising direction for exact algorithms for the KPC.*

Resumo. *O Problema da Mochila com Conflitos (KPC) é uma extensão NP-hard do problema clássico da mochila que introduz restrições disjuntivas entre itens, aumentando significativamente a complexidade dos métodos exatos de resolução. Este trabalho propõe uma heurística de ramificação, denominada S_i , que incorpora atributos topológicos do grafo de conflitos, como o grau dos vértices, ao processo de decisão do algoritmo Branch-and-Bound. A heurística combina um fator topológico com um termo econômico adaptativo sensível à densidade do grafo de conflitos. Experimentos computacionais conduzidos em duas camadas metodológicas distintas demonstram que a abordagem proposta reduz, em média, 44,31% o número de nós explorados em comparação com a regra clássica de ramificação Most Fractional. Os resultados confirmam a eficácia do uso de informações topológicas para induzir podas mais precoces, preservando a otimalidade das soluções, e sugerem que estratégias de ramificação orientadas pela estrutura do grafo constituem uma direção promissora para algoritmos exatos aplicados ao KPC.*

1. Introdução

A Otimização Combinatória desempenha um papel central na resolução de problemas de decisão em engenharia e logística. Entre seus modelos mais conhecidos, o Problema da Mochila (*Knapsack Problem*) destaca-se por representar problemas de alocação de recursos sob restrições de capacidade. Contudo, aplicações reais frequentemente exigem que a seleção de recursos respeite relações de exclusividade, dando origem ao Problema da Mochila com Conflitos (*Knapsack Problem with Conflicts - KPC*), uma extensão do

modelo clássico que aumenta significativamente a complexidade computacional devido à natureza *NP-hard* do problema.

A resolução exata do *KPC* baseia-se, predominantemente, em algoritmos de *Branch-and-Bound*. Nesses métodos, a eficiência da busca é determinada pela regra de ramificação (*branching rule*), responsável por decidir qual variável fracionária será fixada em cada nó da árvore. *Solvers* comerciais de última geração empregam estratégias como o *Pseudo-cost Branching*, enquanto a literatura acadêmica frequentemente recorre a critérios mais simples, como o *Most Fractional*, como ponto de comparação. Ambos, contudo, permanecem "cegos" à topologia de conflitos que caracteriza o *KPC*. Ao desconsiderarem a estrutura do grafo de incompatibilidade, essas estratégias podem explorar ramos redundantes, retardando a convergência para a solução ótima.

Neste trabalho, propomos uma estratégia de ramificação heurística baseada na estrutura topológica do grafo de conflitos, denominada S_i . Esta heurística integra, em uma única formulação, o grau de conflito de cada item, sua eficiência econômica (lucro/peso) e um expoente adaptativo sensível à densidade global do grafo de conflitos. A hipótese central é que, ao priorizar itens que atuam como "hubs" de conflito ou que possuem alta eficiência econômica, o algoritmo de busca pode induzir podas mais precoces, reduzindo o esforço computacional sem comprometer a exatidão.

Para validar a eficácia da abordagem S_i , este artigo apresenta um estudo rigoroso em duas camadas experimentais. Na primeira, avalia-se a heurística em ambiente isolado, demonstrando a redução bruta no tamanho da árvore de busca. Na segunda, confronta-se S_i com os mecanismos de otimização de *solvers* comerciais em um ambiente de produção simulado, evidenciando o custo relativo de operar sem os mecanismos dinâmicos nativos do *solver*. Os resultados obtidos evidenciam a eficiência da topologia capturada por S_i na redução do esforço computacional, sem comprometer a qualidade das soluções encontradas.

As principais contribuições deste trabalho podem ser resumidas nos seguintes pontos:

- Proposição de uma nova heurística de ramificação baseada em informações topológicas do grafo de conflitos;
- Desenvolvimento de um score adaptativo que combina fatores estruturais e econômicos em função da densidade do grafo;
- Avaliação experimental em ambiente isolado, permitindo medir o impacto puro da estratégia de ramificação;
- Investigação do comportamento da heurística em ambiente comercial utilizando o CPLEX;
- Análise da influência da densidade do grafo sobre a eficiência da estratégia proposta, com aprofundamento particular no ambiente isolado da Camada 1.

Este artigo está organizado da seguinte forma: a Seção 2 revisa a fundamentação teórica; a Seção 3 formaliza a heurística proposta; a Seção 4 detalha a metodologia expe-

rimental; a Seção 5 discute os resultados comparativos; e, finalmente, a Seção 6 apresenta as conclusões e os trabalhos futuros.

2. Fundamentação Teórica

Esta seção revisa os conceitos fundamentais necessários para a compreensão do Problema da Mochila com Conflitos (*Knapsack Problem with Conflicts - KPC*) e os mecanismos exatos de resolução baseados em árvores de busca, estabelecendo a base teórica para a heurística proposta neste trabalho.

2.1. O Problema da Mochila 0 – 1 Clássico

O Problema da Mochila 0 – 1 (*Knapsack Problem - KP*) é um dos modelos mais estudados na área de Otimização Combinatória. Conforme a obra de Martello e Toth [Martello and Toth 1990], o problema consiste em selecionar um subconjunto de itens a partir de um conjunto disponível, maximizando o lucro total sem exceder a capacidade máxima da mochila.

Matematicamente, dado um conjunto de n itens, onde cada item $i \in \{1, \dots, n\}$ possui um lucro $c_i > 0$ e um peso $w_i > 0$, e uma mochila de capacidade W , introduz-se a variável de decisão binária $x_i \in \{0, 1\}$, onde $x_i = 1$ representa que o item i foi selecionado e $x_i = 0$ caso contrário. O problema é formulado como:

$$\max \sum_{i=1}^n c_i x_i \quad (1)$$

$$\text{sujeito a: } \sum_{i=1}^n w_i x_i \leq W \quad (2)$$

Apesar de sua formulação simples e sua natureza *NP-Hard*, o *KP* possui uma estrutura que permite a resolução eficiente de muitas instâncias práticas, servindo como base para a modelagem de aplicações complexas e surgindo frequentemente como subproblema em rotinas de planos de corte [Gu et al. 1998] e geração de colunas.

2.2. O Problema da Mochila com Conflitos (*KPC*)

Apesar de ser amplamente aplicável, o modelo clássico assume que a seleção de um item independe dos demais. Em diversos cenários reais, restrições lógicas impedem que certos pares de itens sejam selecionados juntamente. O Problema da Mochila com Conflitos surge como uma generalização que integra o *KP* ao Problema do Conjunto Independente (*Independent Set*).

Formalizado por Pferschy e Schauer [Pferschy and Schauer 2009], o *KPC* introduz um grafo de incompatibilidade $G = (V, E)$, no qual o conjunto de vértices $V = \{1, \dots, n\}$ representa os itens e as arestas E indicam os conflitos. A existência de uma aresta $(i, j) \in E$ impõe a restrição disjuntiva:

$$x_i + x_j \leq 1 \quad \forall (i, j) \in E \quad (3)$$

$$x_i \in \{0, 1\}, \quad i = 1, \dots, n \quad (4)$$

Conforme demonstrado por Pferschy e Schauer [Pferschy and Schauer 2009], essa restrição torna o problema computacionalmente mais exigente. Em particular, as restrições de conflito induzem complexidades poliédricas associadas ao Empacotamento de Conjuntos (*Set Packing*), cuja teoria foi explorada por Padberg [Padberg 1977].

2.3. Resolução Exata e Estratégias de Ramificação

Para a resolução exata de formulações inteiras como o *KPC*, o método *Branch-and-Bound* (*B&B*) permanece como a principal estratégia. Como mostrado por Wolsey [Wolsey 2021], o algoritmo particiona o espaço de soluções construindo uma árvore de busca, avaliando os limites superiores (*bounds*) através da relaxação linear, onde a restrição $x_i \in \{0, 1\}$ é relaxada para $x_i \in [0, 1]$, para podar caminhos subótimos.

A literatura tem focado esforços no desenvolvimento de algoritmos exatos customizados para o *KPC*. Trabalhos como os de Bettinelli, Cacchiani e Malaguti [Bettinelli et al. 2017] e o algoritmo combinatorial de Coniglio, Furini e San Segundo [Coniglio et al. 2021] evidenciam que a topologia do grafo $G = (V, E)$ dita a eficiência da busca. No entanto, em *solvers* comerciais, a regra de ramificação que decide qual variável fracionária x_i será forçada a 0 ou 1 é dada por padrões matemáticos. A regra *Most Fractional*, que prioriza variáveis cujo valor relaxado se aproxima de 0.5, negligencia as informações topológicas de conflito, gerando árvores de busca com número excessivo de nós.

2.4. Influência da Topologia do Grafo na Ramificação

No Problema da Mochila com Conflitos, a qualidade de uma decisão de ramificação está diretamente relacionada à estrutura do grafo de incompatibilidade. Diferentemente do Problema da Mochila clássico, no qual a seleção de um item afeta apenas a restrição de capacidade, no *KPC* a decisão sobre uma variável pode alterar simultaneamente a viabilidade de diversos outros itens conectados por arestas de conflito.

Em algoritmos tradicionais de *Branch-and-Bound*, critérios de seleção de variáveis são frequentemente baseados exclusivamente na solução da relaxação linear. Regras como *Most Fractional* priorizam variáveis cujo valor relaxado se aproxima de 0.5, interpretando esse comportamento como um indicador de incerteza. Embora essa estratégia seja eficaz em diversos problemas de programação inteira, ela não incorpora informações estruturais presentes no grafo de conflitos.

A densidade do grafo também influencia esse comportamento: em grafos esparsos, características econômicas dos itens tendem a ser mais discriminativas entre candidatos à ramificação, enquanto em grafos densos a própria estrutura de conflitos passa a dominar essa distinção. A Seção 3.1 formaliza essa observação por meio de um expoente adaptativo sensível à densidade do grafo.

Dessa forma, a qualidade de uma regra de ramificação no *KPC* não pode ser avaliada apenas pelo grau de incerteza associado à variável. Informações topológicas, como conectividade e influência estrutural dos vértices, podem fornecer indícios adicionais sobre o potencial de uma decisão em reduzir a árvore de busca. Dessa forma, a incorporação de características do grafo de conflitos surge como uma alternativa natural para orientar o processo de ramificação em direção a decisões potencialmente mais informativas.

3. Estratégia de Ramificação Baseada em Conflitos

Como discutido na seção anterior, a eficiência de um algoritmo de *Branch-and-Bound* na resolução do *KPC* é impactada pela ordem em que as variáveis são escolhidas para a ramificação. As abordagens tradicionais, focadas na incerteza matemática (como a distância do valor fracionário para 0.5), perdem a oportunidade de explorar a topologia de incompatibilidade do problema. Neste trabalho, propomos uma estratégia de ramificação heurística baseada na estrutura do próprio grafo de conflitos.

3.1. Definição do Score de Ramificação

Seja $\bar{x}$ o vetor de solução da relaxação linear em um determinado nó da árvore de busca. Definimos o conjunto de variáveis fracionárias elegíveis para a ramificação neste nó como:

$$F = \{i \in V : \bar{x}_i \in (0, 1)\} \quad (5)$$

O objetivo da heurística proposta é selecionar o item $i \in F$ que, ao ter seu valor fixado em 0 ou 1, provoque a maior degradação do limite superior (*upper bound*) da relaxação linear, induzindo podas (*pruning*) precoces. Para que a avaliação seja invariante à escala das instâncias (lucros excessivamente altos ou graus muito variados), aplicamos a normalização *Min-Max* no intervalo $[0, 1]$ para o grau de conflito, o lucro e o peso de cada item i , denotados respectivamente por $\hat{d}_i$, $\hat{c}_i$ e $\hat{w}_i$.

Propomos a função *score* S_i definida como:

$$S_i = (1 + \hat{d}_i) \cdot \left(\frac{\hat{c}_i}{\hat{w}_i^{\max(0.5, 1-\Delta)}} \right), \quad \forall i \in F \quad (6)$$

onde Δ é a densidade do grafo de conflitos, ou seja, a razão entre o número de arestas existentes e o número máximo de arestas possíveis.

A formulação de S_i é sustentada por dois mecanismos estruturais:

- **Fator Topológico** ($1 + \hat{d}_i$): Um vértice isolado ($\hat{d}_i = 0$) recebe um multiplicador 1.0, sendo avaliado estritamente pelo seu mérito econômico. Vértices com alta conectividade recebem multiplicadores próximos a 2.0, sinalizando urgência para ramificação, visto que fixá-los induzirá múltiplas alterações na árvore devido a restrição disjuntiva.
- **Termo Econômico Adaptativo** ($\hat{c}_i / \hat{w}_i^{\max(0.5, 1-\Delta)}$): A razão lucro/peso normalizada prioriza itens eficientes, impedindo que o fator topológico esmague isoladamente o objetivo econômico da mochila. O expoente sobre $\hat{w}_i$ modula a intensidade desse termo de acordo com a densidade Δ : em grafos esparsos ($\Delta \approx 0$), o expoente aproxima-se de 1, preservando a relação lucro/peso original. Em grafos densos, essa atenuação é motivada por uma análise empírica anterior (KPCG-Bench) [Santos 2026], que identificou que a correlação financeira entre itens perde poder preditivo sobre o comportamento da árvore de busca conforme a densidade de conflitos aumenta, a topologia do grafo passa a dominar esse comportamento. O limite inferior de 0.5 atua como trava de segurança, garantindo que o termo econômico nunca seja completamente eliminado, apenas atenuado.

3.2. Critério de Seleção

A regra de ramificação proposta funciona como um interceptador (*callback*) no algoritmo exato. Em cada nó da árvore onde $F \neq \emptyset$, o algoritmo calcula o *score* S_i para todas as variáveis fracionárias. A variável selecionada para gerar os novos ramos do *Branch-and-Bound* será a de maior *score*:

$$i^* = \arg \max_{i \in F} \{S_i\} \quad (7)$$

Em caso de múltiplos itens maximizarem a Equação (7), o critério de desempate recorre à regra *Most Fractional* padrão, selecionando o item de maior S_i cujo valor $\bar{x}_i$ mais se aproxima de 0.5, isto é, que minimiza a distância $|0.5 - \bar{x}_i|$. A formulação de S_i garante que a árvore de busca priorize a resolução dos vértices que mais inflam artificialmente o limite linear, acelerando a convergência do algoritmo exato.

O produto foi adotado como operador de composição pois penaliza multiplicativamente itens de baixo grau ou baixa eficiência, impedindo que uma métrica elevada compense artificialmente a outra.

3.3. Complexidade Computacional da Heurística

A aplicação da heurística proposta introduz um custo computacional adicional associado ao cálculo do *score* S_i para as variáveis fracionárias presentes em cada nó da árvore de *Branch-and-Bound*. Entretanto, esse custo é relativamente pequeno quando comparado ao esforço necessário para resolver a relaxação linear correspondente.

Considere F como um conjunto de variáveis fracionárias em um determinado nó da árvore de busca. Para cada item $i \in F$, o cálculo do *score* S_i depende do grau de conflito normalizado $\hat{d}_i$, do lucro normalizado $\hat{c}_i$, do peso normalizado $\hat{w}_i$ e da densidade global do grafo Δ .

A densidade do grafo é calculada apenas uma vez antes do início da execução do algoritmo. Da mesma forma, os graus dos vértices e as normalizações dos parâmetros dos itens podem ser pré-processados durante a leitura da instância, não sendo necessário recalculá-los ao longo da árvore de busca. Consequentemente, em cada nó, o algoritmo precisa apenas avaliar a Equação 6 para os elementos pertencentes ao conjunto F .

Assim, o custo da seleção da variável de ramificação em um nó é proporcional ao número de variáveis fracionárias, resultando em uma complexidade temporal $O(|F|)$. Como $|F| \leq n$, o pior caso corresponde a $O(n)$, embora, na prática, o número de variáveis fracionárias seja frequentemente muito inferior ao número total de itens.

Além disso, o custo da avaliação do *score* é composto apenas por operações aritméticas elementares, como multiplicações, divisões e exponenciação, cujo impacto computacional é reduzido quando comparado ao tempo necessário para a resolução da relaxação linear em cada nó do *Branch-and-Bound*.

Dessa forma, a heurística proposta busca melhorar a qualidade das decisões de ramificação sem introduzir sobrecarga significativa ao processo de busca. O custo adicional por nó permanece linear no número de variáveis candidatas, enquanto os ganhos potenciais decorrem da redução do número total de nós explorados.

4. Metodologia e Implementação

Para avaliar a eficácia do score proposto (S_i), os experimentos foram planejados em duas camadas metodológicas distintas. Os testes foram implementados em linguagem C++, utilizando a API da biblioteca comercial IBM ILOG CPLEX. O principal objetivo desta divisão é isolar o impacto da regra de ramificação da influência dos diversos mecanismos de aceleração internos do *solver*.

4.1. Camada 1: Ambiente Isolado (Prova de Conceito)

O objetivo desta camada é validar puramente a formulação matemática da heurística, comparando-a de forma direta com a regra de ramificação clássica da literatura, a *Most Fractional*.

Para estabelecer um ambiente de laboratório controlado e neutro, todos os mecanismos de aceleração do CPLEX, incluindo o pré-processamento (*Presolve*), a geração de planos de corte no nó raiz e as heurísticas primais, foram completamente desativados. Ambas as regras de ramificação foram incorporadas ao algoritmo exato por meio de *Legacy Callbacks* estritos.

Esta camada atua como uma prova de conceito matemático: ao desabilitar os mecanismos internos de aceleração do *solver*, força-se o crescimento da árvore de busca. Dessa forma, avalia-se se a incorporação de conhecimento topológico é capaz de reduzir o tamanho bruto da árvore em comparação com uma métrica baseada exclusivamente na incerteza fracionária.

4.2. Camada 2: Ambiente Comercial (Comparação Clássica)

Na segunda fase, o objetivo é avaliar a viabilidade prática da heurística S_i contra o algoritmo *default* do CPLEX em um cenário de produção máximo. Para isso, o CPLEX padrão operou com a totalidade dos seus recursos modernos de resolução ativados, incluindo o pré-processamento (*Presolve*), a geração de cortes na raiz e o seu motor de busca avançada (*Dynamic Search*).

Por outro lado, a implementação da heurística S_i via *Legacy Callbacks* impõe uma limitação técnica: por razões de segurança interna, o CPLEX desativa o *Dynamic Search* automaticamente sempre que um *callback* de controle é registrado, forçando a execução a ocorrer sob a estrutura de árvore estática clássica (*Traditional Branch-and-Cut*).

Optou-se por não nivelar as arquiteturas, submetendo a heurística proposta a uma desvantagem intencional. Essa configuração atua como um teste de resistência computacional: o objetivo é investigar até que ponto a informação topológica agregada na regra de ramificação consegue sobreviver e competir contra as heurísticas e os pseudocustos nativos de um dos *solvers* comerciais mais avançados disponíveis, operando em sua plena capacidade.

5. Resultados e Discussão

Nesta seção, apresentam-se os resultados computacionais obtidos através da metodologia descrita anteriormente. A análise é dividida de acordo com as duas camadas experimentais, permitindo uma interpretação crítica sobre o desempenho da heurística proposta

(S_i) tanto em ambiente controlado quanto em cenário comercial. Antes de discutir desempenho, cabe destacar um resultado comum a ambas as camadas: em nenhuma das instâncias testadas, sob nenhuma das quatro configurações avaliadas, a heurística S_i comprometeu a otimalidade da solução encontrada. A paridade de lucro entre S_i e as regras de comparação foi de 100%, confirmando que os ganhos e perdas discutidos a seguir referem-se exclusivamente ao esforço computacional, e não à qualidade da solução.

5.1. Análise do Ambiente Isolado (Camada 1)

Na primeira camada, observou-se que a incorporação do conhecimento topológico na regra de ramificação resulta em uma redução consistente e significativa no esforço computacional. Em média, a heurística S_i alcançou uma redução de 44,31% no número de nós explorados e 34,90% no tempo total de processamento em comparação com a regra clássica *Most Fractional*.

Estes dados evidenciam que, ao priorizar itens com maior grau de conflito e eficiência econômica simultaneamente, o *solver* é capaz de realizar podas mais precoces e eficazes. O gráfico da Figura 1 ilustra que a eficácia da heurística é particularmente pronunciada em instâncias de baixa complexidade ($\Delta \approx 20\%$), independentemente do número de itens, onde o "desvio" da estratégia clássica se torna um diferencial competitivo na contenção do crescimento exponencial da árvore de busca. Esse padrão é consistente com a motivação original da Equação (6): em grafos esparsos, a informação econômica (lucro/peso) tem maior poder discriminativo entre os itens, permitindo que S_i direcione a busca de forma mais eficaz que o critério puramente fracionário do *Most Fractional*.

Conforme a densidade do grafo aumenta, contudo, esse diferencial se atenua progressivamente: nas instâncias com $\Delta \geq 80\%$, a redução média no número de nós cai para a faixa de 15% a 25%. Esse comportamento confirma a premissa de que, em grafos densos, a topologia por si só já restringe fortemente o espaço de soluções viáveis, reduzindo o espaço para que a informação econômica adicional discrimine entre os candidatos à ramificação, e, por extensão, reduzindo a margem de melhoria disponível para qualquer heurística que dependa dessa informação. A Tabela 1 consolida esse padrão, agrupando as 60 instâncias testadas em três faixas de densidade.

Faixa de densidade	Inst.	Red. média de nós	Red. média de tempo
Esparso ($\Delta \approx 20\%$)	15	72, 7%	71, 2%
Médio ($\Delta \approx 40\text{--}60\%$)	30	54, 5%	51, 3%
Denso ($\Delta \approx 80\%$)	15	24, 5%	8, 8%

Tabela 1. Resumo da Camada 1 por faixa de densidade do grafo de conflitos.

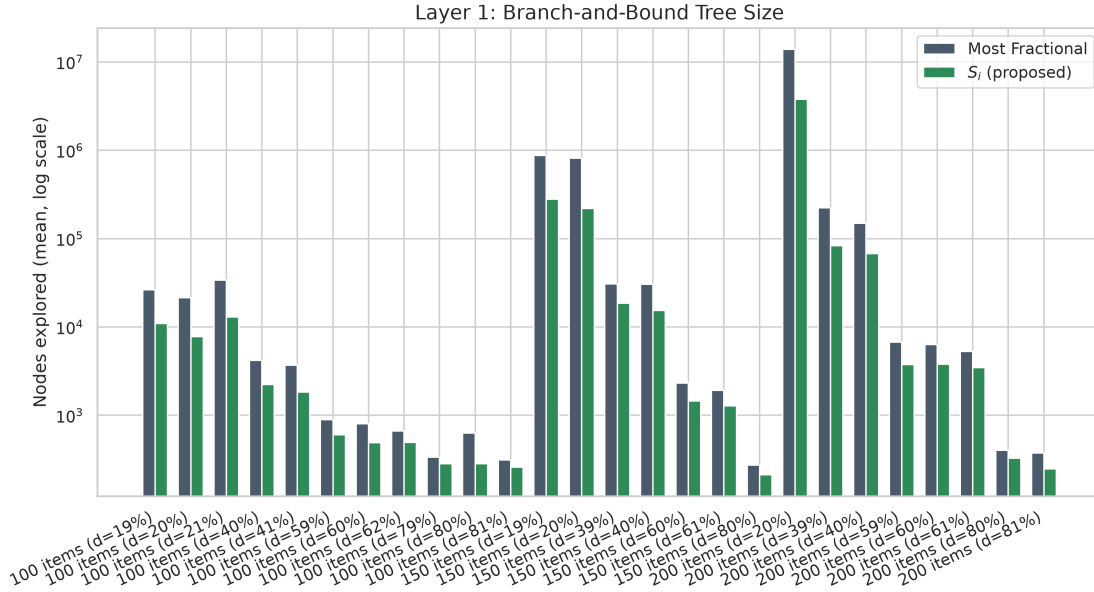


Figura 1. Número de nós explorados - Camada 1.

5.2. Análise do Ambiente Comercial (Camada 2)

Na segunda camada, o confronto entre a heurística S_i (via *Legacy Callbacks*) e o CPLEX padrão (operando em modo *full* com *Dynamic Search*) revelou o comportamento da heurística frente a mecanismos de otimização de alto nível.

Como previsto, a desativação forçada do *Dynamic Search*, devido à limitação técnica associada ao uso de *callbacks* de controle no CPLEX, impôs uma desvantagem severa à nossa implementação. Os resultados indicam um aumento no tempo de CPU e no número de nós explorados quando comparados ao modo de operação otimizado do *solver* comercial. A Tabela 2 consolida os resultados métricos de tempo por faixa de densidade.

¹

Faixa de densidade	Inst.	Tempo médio (CPLEX)	Tempo médio (S_i)
Esparso ($\Delta \approx 20\%$)	15	0,42s	1,67s
Médio ($\Delta \approx 40\text{--}60\%$)	30	1,73s	7,28s
Denso ($\Delta \approx 80\%$)	15	1,13s	7,53s

Tabela 2. Resumo da Camada 2 por faixa de densidade do grafo de conflitos (tempo de CPU).

Um exame mais detalhado dos dados, porém, sugere que essa desvantagem não é uniforme. Em instâncias de baixa densidade ($\Delta \approx 20\%$), a penalidade relativa de tempo, embora ainda alta, é proporcionalmente menor do que em instâncias de alta densidade

¹Para o número de nós, a métrica de redução percentual não é aplicável em diversas categorias: o CPLEX padrão resolve a maior parte das instâncias de densidade baixa e média sem explorar nenhum nó (a relaxação do nó raiz já é suficiente, frequentemente em conjunto com os cortes gerados ali), tornando o denominador da razão igual a zero. Por esse motivo, a Tabela 2 reporta apenas a métrica de tempo, que permanece bem definida em todos os casos.

($\Delta \approx 80\%$), como evidenciado na Tabela 2. Esse padrão indica que parte do custo observado na Camada 2 não decorre apenas da ausência do *Dynamic Search*, mas também do *overhead* intrínseco de manter um *callback* de controle ativo a cada nó da árvore, *overhead* que se torna proporcionalmente mais relevante quando a árvore de busca é pequena. Esse efeito é particularmente visível nas instâncias em que tanto o CPLEX padrão quanto S_i resolvem o problema sem qualquer ramificação: mesmo nesses casos, em que as duas abordagens exploram zero nós, S_i ainda apresenta tempo de execução superior, o que isola o custo do *callback* em si como um componente relevante da degradação observada, independentemente da qualidade das decisões de ramificação que ele toma.

Esta análise é fundamental para avaliar se a heurística S_i preserva sua eficácia relativa mesmo na ausência dos mecanismos dinâmicos do *solver*. Mesmo sem esses mecanismos, a heurística mantém-se robusta e exata, sem apresentar comportamento errático. A degradação de desempenho observada é, portanto, melhor compreendida como o custo de duas fontes distintas e cumulativas: a perda do *Dynamic Search* em si, e o *overhead* estrutural de qualquer *callback* de controle, e não como uma falha da lógica de ramificação proposta, cujo mérito relativo já foi estabelecido na Camada 1.

Ainda assim, este cenário permite observar, de forma indireta, a magnitude da contribuição do *Dynamic Search* e do *Pseudo-cost Branching* para o desempenho do CPLEX: a diferença de tempo entre a configuração padrão do *solver* nas Camadas 1 e 2, para uma mesma instância, funciona como um indicativo da economia computacional que esses mecanismos comerciais proporcionam quando habilitados. Isso sugere que uma integração futura entre a informação topológica capturada por S_i e as estratégias de busca dinâmica nativas do *solver*, por exemplo, por meio de *Generic Callbacks*, que não impõem a mesma restrição de desativação do *Dynamic Search*, constitui uma direção promissora para trabalhos futuros.

5.3. Limitações

Embora os resultados obtidos sejam promissores, algumas limitações devem ser consideradas. A implementação da heurística por meio de *Legacy Callbacks* impõe a desativação do *Dynamic Search* do CPLEX, dificultando uma comparação direta entre a estratégia proposta e o *solver* operando em sua configuração padrão.

Além disso, as instâncias utilizadas foram geradas segundo um conjunto controlado de densidades de conflito. Embora essa metodologia permita analisar o impacto estrutural do grafo, estudos adicionais em *benchmarks* amplamente utilizados na literatura podem ampliar a validade externa dos resultados.

Por fim, a função *score* proposta utiliza parâmetros fixos, não adaptativos, que podem ser aprimorados por técnicas de aprendizado de máquina ou estratégias dinâmicas de ajuste.

6. Conclusão

Este trabalho apresentou uma nova estratégia de ramificação heurística, denominada S_i , para a resolução do Problema da Mochila com Conflitos (KPC). A hipótese central da pesquisa, segundo a qual a incorporação de informações topológicas do grafo de conflitos pode melhorar a eficiência do processo de ramificação, foi confirmada pelos resultados experimentais obtidos.

Os experimentos conduzidos em duas camadas metodológicas evidenciaram a capacidade da heurística em direcionar a busca para regiões estruturalmente relevantes do grafo de incompatibilidade. No ambiente isolado, a abordagem proposta reduziu, em média, 44, 31% do número de nós explorados em comparação com a regra clássica *Most Fractional*, demonstrando que a utilização de informações topológicas pode induzir podas mais precoces e reduzir significativamente o esforço computacional.

No ambiente comercial, a implementação baseada em *Legacy Callbacks* impôs restrições técnicas associadas à natureza dessa interface e ao custo computacional dos mecanismos de controle. Ainda assim, a heurística manteve a otimalidade das soluções em todas as instâncias avaliadas, evidenciando a robustez da formulação proposta.

A análise de complexidade mostrou que o cálculo do *score* apresenta custo temporal linear em relação ao número de variáveis fracionárias presentes em cada nó, isto é, $O(|F|)$. Dessa forma, o custo adicional introduzido pela heurística permanece reduzido quando comparado ao esforço necessário para a resolução das relaxações lineares, indicando que os ganhos observados decorrem principalmente da qualidade das decisões de ramificação.

Como perspectivas futuras, destacam-se a migração da implementação para mecanismos modernos de *Generic Callbacks*, permitindo a coexistência com o *Dynamic Search*, a incorporação de técnicas de aprendizado de máquina para ajustar dinamicamente os parâmetros da função S_i ; e a investigação da aplicabilidade da heurística em outros problemas de otimização que utilizam grafos de conflito.

Em resumo, os resultados obtidos reforçam a importância da exploração de informações estruturais e topológicas no desenvolvimento de estratégias de ramificação para problemas de otimização combinatória, sugerindo que abordagens orientadas pela estrutura do grafo constituem uma direção promissora para o aprimoramento de algoritmos exatos aplicados ao KPC.

Referências

- Bettinelli, A., Cacchiani, V., and Malaguti, E. (2017). A branch-and-bound algorithm for the knapsack problem with conflict graph. *INFORMS Journal on Computing*, 29:457–473.
- Coniglio, S., Furini, F., and San Segundo, P. (2021). A new combinatorial branch-and-bound algorithm for the knapsack problem with conflicts. *European Journal of Operational Research*, 289(2):435–455.
- Gu, Z., Nemhauser, G. L., and Savelsbergh, M. W. P. (1998). Lifted cover inequalities for 0-1 integer programs: Computation. *INFORMS Journal on Computing*, 10(4):427–437.
- Martello, S. and Toth, P. (1990). *Knapsack Problems: Algorithms and Computer Implementations*. Wiley-Interscience Series in Discrete Mathematics and Optimization. John Wiley & Sons, Chichester.
- Padberg, M. W. (1977). On the complexity of set packing polyhedra. In Hammer, P., Johnson, E., Korte, B., and Nemhauser, G., editors, *Studies in Integer Programming*, volume 1 of *Annals of Discrete Mathematics*, pages 421–434. Elsevier.

- Pferschy, U. and Schauer, J. (2009). The knapsack problem with conflict graphs. *Journal of Graph Algorithms and Applications*, 13(2):233–249.
- Santos, M. K. T. (2026). Kpcg-bench: Um benchmark sintético e preditor de estratégia via aprendizado de máquina para o knapsack problem with conflicts. Trabalho não publicado, desenvolvido na disciplina de Ciência de Dados, Instituto de Computação, Universidade Federal de Alagoas (UFAL).
- Wolsey, L. A. (2021). *Integer Programming*. John Wiley & Sons, Hoboken, 2 edition.